



Работа с Docker. Часть 4.

Продвинутая работа с Docker. Сеть и тома.

Этап 1. Сеть по умолчанию.

Войдите на виртуальную машину с Docker.

```
vagrant ssh
```

Давайте глянем на сеть. Пока у нас сеть по умолчанию.

```
docker network ls
```

Мы можем запросить дополнительную информацию для одной из существующих сетей.

Например, для bridge:

```
docker network inspect bridge
```

Мы можем увидеть диапазон IP-адресов, привязку на существующий сетевой адаптер в системе и т. д.

Запустим два контейнера на основе alpine

```
docker container run -dt --name alp1 alpine sleep 1d
```

```
docker container run -dt --name alp2 alpine sleep 1d
```

Теперь посмотрите на информацию о сети

```
docker network inspect bridge
```

Мы видим два контейнера с их IP-адресами.

В качестве альтернативы мы можем выполнить следующую команду, причем взять ту информацию, которая нам нужна.

```
docker network inspect bridge -f '{{range .Containers}}{{.Name}},  
{{println .IPv4Address}}{{end}}'
```

Подключимся к первому контейнеру

```
docker container exec -it alp1 sh
```

И проверим его IP-адрес

```
ip a
```

Теперь попробуйте пропинговать сайт.

```
ping -c 4 ya.ru
```

Сайты мы пингуем. А как насчет связи с другим контейнером (**alp2**) по его имени?

```
ping -c 4 alp2
```

Хм, не работает? А по IP-адресу?

```
ping -c 4 <введите ip>
```

Сработал?

Так как наши контейнеры могут общаться по адресу? Или по имени?



Облачные платформы и виртуализация

Закрываем сеанс в контейнере

```
exit
```

Останавливаем и удаляем контейнеры

```
docker container rm --force alp1 alp2
```

Этап 2. Пользовательская сеть.

Давайте создадим наш собственный мост с именем mybridge.

```
docker network create -d bridge --subnet 10.0.0.0/24 mybridge
```

И проверьте, появляется ли в списке

```
docker network ls
```

Мы можем запросить дополнительную информацию о сети, которую мы только что создали:

```
docker network inspect mybridge
```

Давайте создадим два контейнера и разместим их в нашей новой сети.

Мы снова воспользуемся alpine.

```
docker container run -dt --name co1 --network mybridge alpine sleep 1d
```

```
docker container run -dt --name co2 --network mybridge alpine sleep 1d
```

Теперь проверим нашу сеть

```
docker network inspect mybridge
```

Мы видим два контейнера с их адресами.

Теперь мы можем войти в первый контейнер и проверить его сетевое подключение.

```
docker container exec -it co1 sh
```

И проверим его IP-адрес

```
ip a
```

Теперь попробуйте пропинговать сайт.

```
ping -c 4 ya.ru
```

Сайты мы пингуем. А как насчет связи с другим контейнером (**co2**) по его имени?



```
ping -c 4 co2
```

Работает? А по адресу?

```
ping -c 4 <введите ip>
```

Работает? Как вы думаете почему? Подумайте...

...
...
...
...

Итак, наши контейнеры могут общаться как по адресу, так и по имени
Для этого мы и создавали пользовательскую сеть на базе моста.
Закрываем сеанс в контейнере

```
exit
```

Останавливаем и удаляем контейнеры

```
docker container rm --force co1 co2
```

И удаляем сеть

```
docker network rm mybridge
```

Этап 3. Тома.

Мы можем обмениваться данными с контейнерами несколькими способами.

Во-первых, мы создадим контейнер, к которому мы прикрепим том на лету (локальная папка, доступная на хосте **Docker**).

Давайте сначала проверим, есть ли какие-нибудь тома:

```
docker volume ls
```

Нет, нет никаких

Теперь создайте контейнер

```
docker container run -it -v /test-vol --name c1 ubuntu /bin/bash
```

Теперь у нас должна быть папка **/test-vol** внутри контейнера.

```
ls -l /
```

Давайте выйдем из контейнера с помощью **Ctrl + P** и **Ctrl + Q**



Проверьте, есть ли какие-либо изменения:

```
docker volume ls
```

Хм, том появился. Это прямой результат команды создания контейнера выше. Создайте второй контейнер, который унаследует том от первого.

```
docker container run -it --volumes-from c1 --name c2 ubuntu /bin/bash
```

Снова выходим из контейнера с помощью **Ctrl + P** и **Ctrl + Q**. И обратно к первому (**c1**)

```
docker container attach c1
```

И создайте текстовый файл в прикрепленной папке к тому

```
echo 'Hi from C1!' >> /test-vol/file.txt
```

Выйдите снова с **Ctrl + P** и **Ctrl + Q** и подключитесь ко второму (**c2**)

```
docker container attach c2
```

Проверьте содержимое файла

```
cat /test-vol/file.txt
```

И добавьте к нему текст

```
echo 'C2 is here!' >> /test-vol/file.txt
```

Проверяем, что текст записан в файл

```
cat /test-vol/file.txt
```

Выходим, останавливаем контейнер (**c2**)

```
exit
```

Теперь присоединяемся к первому контейнеру (**c1**)

```
docker container attach c1
```

Проверьте содержимое файла

```
cat /test-vol/file.txt
```



Облачные платформы и виртуализация

И выйдите из контейнера (c1)

```
exit
```

Убедимся, что оба контейнера остановлены

```
docker container ls
```

Теперь мы можем запустить третий контейнер (c3), который унаследует том от c1.

```
docker container run -it --volumes-from c1 --name c3 ubuntu /bin/bash
```

Проверьте содержимое файла

```
cat /test-vol/file.txt
```

И добавьте к нему строку

```
echo 'C3 joined the party!' >> /test-vol/file.txt
```

Выйдите из контейнера с помощью **Ctrl + P** и **Ctrl + Q**

Затем снова запустите контейнер c1

```
docker container start -i c1
```

И проверьте содержимое файла

```
cat /test-vol/file.txt
```

Снова выйдите с помощью **Ctrl + P** и **Ctrl + Q**

Давайте посмотрим запущенные контейнеры

```
docker container ls
```

Чтобы проверить, где данные хранятся, выполните

```
docker container inspect c1 | grep -i source
```

То же самое должно быть и для контейнера c3

Теперь посмотрим папку на нашем хосте **Docker**, где хранятся данные. Замените id на свой:

```
sudo ls -l /var/lib/docker/volumes/<volume-id>/_data
```

Кроме того, мы можем запросить список томов с помощью



```
docker volume ls
```

Подробная информация, замените id на свой:

```
docker volume inspect <volume-id>
```

Теперь мы можем проверить содержимое файла с хоста.

Измените на пользователя **root** с помощью

```
sudo -i
```

Перейдите к папке:

```
cd /var/lib/docker/volumes/<volume-id>/_data
```

Проверьте содержимое файла

```
cat file.txt
```

Добавьте к нему текст

```
echo 'Changed on the host' >> file.txt
```

Затем выйдите из сеанса

```
exit
```

И вернитесь в один из контейнеров, например, c1)

```
docker container attach c1
```

Давайте еще раз проверим файл

```
cat /test-vol/file.txt
```

Закройте сессию в контейнере последовательностью **Ctrl + P** и **Ctrl + Q**

В качестве последнего шага мы можем остановить оба запущенных контейнера.

```
docker container stop c1 c3
```

И удалить том

```
docker volume rm <volume-id>
```

Упс, мы должны сначала удалить остановленные контейнеры. Итак, давайте сделаем это

```
docker container rm c1 c2 c3
```



Облачные платформы и виртуализация

Теперь мы также можем удалить том (*повторите команду из предыдущей попытки удаления тома*)

Этап 4. Подготовленная папка.

Давайте сначала создадим папку, например **web**

```
mkdir web
```

И добавьте простой **index.html** файл в нем

```
echo '<h2>Hello from a Docker Volume</h2>' > web/index.html
```

Теперь мы можем запустить контейнер с прикрепленной к нему папкой.

```
docker container run -d -p 8080:80 --name co-apache -v $(pwd)/web:/var/www/html php:8.0-apache
```

Откройте браузер на своем хосте и введите следующий URL-адрес:
<http://192.168.99.100:8080>.

Если у вашего экземпляра **Docker** другой IP-адрес, используйте его.

Подключимся:

```
docker container exec -it co-apache bash
```

Глянем папку:

```
ls -al
```

И файл:

```
cat index.html
```

Изменим файл, находящийся внутри контейнера

```
echo '<br />Changed inside the container' >> index.html
```

Если мы обновим вкладку браузера, мы увидим разницу

Выходим из сеанса контейнера

```
exit
```

Давайте изменим файл на хосте **Docker**

```
echo '<br /><br />Updated on the host' >> web/index.html
```

Если мы обновим вкладку браузера, мы увидим разницу

Проверим, есть ли том для смонтированной папки

```
docker volume ls
```



Похоже, что нет. Это важное отличие по сравнению с предыдущим примером.

Давайте остановим контейнер

```
docker container stop co-apache
```

Этап 5. Выделенный том.

Давайте заранее создадим том с помощью

```
docker volume create vol-1 --label mode=prod
```

И еще один

```
docker volume create vol-2
```

Глянем список томов

```
docker volume ls
```

Изучите детали тома

```
docker volume inspect vol-1
```

Использовать название тома намного проще по сравнению с тем, что мы видели ранее

Мы можем отфильтровать список томов по метке

```
docker volume ls -f label=mode=prod
```

И отформатируйте список томов

```
docker volume ls --format "{{.Name}}: {{.Driver}}: {{.Mountpoint}}"
```

Мы можем создать файл прямо в томе

```
echo '<h2>Volume created with <u>docker volume create</u></h2>' | \  
sudo tee /var/lib/docker/volumes/vol-1/_data/index.html
```

Теперь мы готовы запустить новый контейнер с подключенным томом

```
docker container run -d -p 8080:80 --name co-apache1 -v vol-1:/var/www/html php:8.0-apache
```

Откройте браузер на своем хосте и введите следующий URL-адрес:

<http://192.168.99.100:8080>.

Если у вашего экземпляра **Docker** другой IP-адрес, используйте его.



Остановите контейнер

```
docker container stop co-apache1
```

Этап 6. Контейнеры томов.

С помощью метода, который мы рассмотрим сейчас, мы стремимся не только к возможности обмена данными, но и к меньшему занимаемому пространству.

Давайте создадим контейнер

```
docker container create -v /con-data --name con-store alpine /bin/true
```

И проверим, где хранятся данные

```
docker container inspect con-store | grep -i source
```

Теперь мы можем добавить простой файл **readme.txt**, который будет доступен для других контейнеров.

```
echo 'Read Me File in a Container Volume' | \  
sudo tee /var/lib/docker/volumes/<volume-id>/_data/readme.txt
```

Давайте запустим новый контейнер, который подключен к тому

```
docker container run -d --volumes-from con-store --name alp1 alpine  
sleep 1d
```

Если мы откроем сессию к контейнеру, то увидим, что все ок

```
docker container exec -it alp1 /bin/sh
```

И проверим файл

```
cat /con-data/readme.txt
```

Затем закройте сеанс

```
exit
```

Наконец, мы можем остановить контейнер

```
docker container stop alp1
```



Облачные платформы и виртуализация